Дрейф LLM-агентов: пайплайн статей и фреймворк

Версия 2: переработка по итогам обсуждения, определения по Trustworthy AI Landscape

По материалам ТЗ (`description_rus.txt`),
Trustworthy AI Landscape и обзоров литературы

10 ноября 2026

Кратко

- **ТЗ закрывают Статьи 1 и 2** вместе с MVP фреймворка. В MVP теперь входит минимальная обёртка `monitor`, поэтому пункт ТЗ «независимый модуль» закрыт буквально (раздел 6).
- **Бюджет** (раздел 1). Деньги уходят только на API и подписки агентов-разработчиков: **план — около $6.1 тыс.** на всю серию с резервом 20% (экономный вариант); если появятся деньги, расширение до ∼$12.1 тыс. Основная сетка идёт на DeepSeek V3.2 через API (∼$0.15 за траекторию) и на открытых моделях на своих GPU (Qwen3, GLM-4.5-Air). Закрытая модель (Claude Sonnet 5.5 или GPT-5) — только на подвыборке как опорная точка. Своих 2×H200 хватает на все GPU-работы, но это ∼**122 дня** непрерывной загрузки по плану (171 при расширении); больше половины уходит на RL в Статье 5.
- **Фреймворк** — шесть пакетов на одной спецификации цели. Из них складываются три продукта: оценка до релиза, охрана в рантайме, укрепление моделей (раздел 5).
- **Оговорка о новизне.** Новые ссылки этой версии найдены веб-поиском и проверены только по аннотациям (пометка [по аннотации]). Перед стартом каждой статьи нужен полный поиск на новизну и чтение ближайших работ.

1 Бюджет

Что считаем. Деньги — только API (агенты под тестом, LLM-мониторы, LLM-судьи) и подписки агентов-разработчиков, которые пишут код фреймворка. GPU — свои 2×H200: денег они не стоят, но занимают календарное время, поэтому GPU-работы указаны в *днях непрерывной загрузки обеих карт*. Расчёт воспроизводится скриптом `roadmap_ru/budget.py`: все допущения в нём, таблица ниже им сгенерирована.

Допущения

Параметр	Значение
Состав моделей	Мы не фронтир-лаба, поэтому моделей мало и каждая выбрана с целью: • **китайский открытый фронтир через API** — DeepSeek V3.2: основная рабочая лошадь, ∼$0.15 за траекторию (на 2×H200 не помещается); • **открытые модели на своих GPU** — Qwen3-8B, Qwen3-32B, Qwen3-30B-A3B (MoE) и китайская MoE класса ∼100B (GLM-4.5-Air, FP8 на двух H200); для Статьи 0 плюс гибриды SSM. Здесь же LLM-судья, синтетика и открытые мониторы; • **одна закрытая фронтир-модель** — Claude Sonnet 5.5 (или GPT-5, -20%): только на подвыборке (25% сетки в базовом варианте, 15% в экономном) как опорная точка «закрытый фронтир»; • **Haiku 5.5** — дешёвый монитор через Batch API. Правило: всё, что помещается на свои GPU, гоним на GPU; API — только для моделей, которые не помещаются, и для опорных точек.
Цены API, $ за 1M токенов (вход / выход)	Claude Sonnet 5.5 — 2 / 10; Haiku 5.5 — 0.10 / 0.50 (прайс Anthropic, кэш от 06.10.2026). GPT-5 — 1.25 / 10; DeepSeek V3.2 — 0.26 / 0.38 (агрегаторы и DeepInfra; октябрь–ноябрь 2026). **Перед тратами сверить с прайсами провайдеров.**
Траектория агента	∼50 шагов, контекст растёт до ∼60K токенов: ∼1.5M входных токенов суммарно по шагам и ∼75K выходных (с рассуждениями). Смесь горизонтов 20/50/100 шагов — множитель 1.4
Кэш промпта	∼85% попаданий: вход обходится в ∼30% номинала
Batch API (-50%)	Только для офлайн-оценки готовых траекторий мониторами Claude (Статья 2). Агенты под тестом идут без batch: шаги последовательны
Стоимость одной траектории	Агент: Sonnet 5.5 — $1.65; GPT-5 — $1.31; DeepSeek V3.2 — $0.15; на своих GPU — $0 (0.01–0.03 H100-часа). Пошаговый монитор: Sonnet (batch) — $0.46; DeepSeek — $0.12; Haiku (batch) — $0.02
GPU	H200 ≈ 1.3× H100 на этих задачах; объёмы оценены в H100-часах и переведены в дни на двух картах
План и расширение	**Начинаем с экономного варианта (план):** 60 задач × 8 условий × 2 seed; закрытая модель на 15% сетки; Qwen3-8B, Qwen3-30B-A3B и GLM-4.5-Air на своих GPU; 8 конфигураций сервинга; 3 RL-прогона на 8B; одно место Claude Max. **Расширение**, если появятся деньги: 80 × 10 × 3; закрытая модель на 25% сетки; плюс Qwen3-32B; 12 конфигураций; 4 прогона на 8B; второе место Claude Max. Расширение докупается по статьям, независимо друг от друга

Бюджет по статьям и задачам

Разбивка по задачам дана для плана; для расширения — только итоги по статьям. «<1» — меньше половины дня GPU.

Статья и задача	План: API, $	План: GPU, дни	Расшир.: API, $	Расшир.: GPU, дни
Статья 0: пробы по архитектурам	**0**	**15**	0	26
Траектории 4 открытых моделей (dense, MoE, гибрид SSM) со случайным d	—	4		
Извлечение активаций и обучение проб	—	3		
Дообучение: LoRA/SFT и RL против пробы	—	9		
Статья 0′: латентный канал	**236**	**11**	354	17
Передача дрейфа: текст против KV-cache и активаций (стенд на готовых C2C/KVComm, 2–3 модели Qwen3)	—	5		
RL-отправитель (латентный сговор)	—	6		
Текстовый LLM-монитор как базовая линия (DeepSeek V3.2)	236	—		
Статья 1: среда и бенчмарк	**822**	**1**	2 608	5
Пилоты и калибровка доз на DeepSeek V3.2 и Qwen3-8B (50% сетки)	98	<1		
Основная сетка: DeepSeek V3.2 через API, 960 прогонов	196	—		
Основная сетка на своих GPU: 3 открытые модели (Qwen3, GLM-4.5-Air)	—	1		
Закрытая модель (Claude Sonnet 5.5) на 15% сетки	333	—		
Абляция каркасов (Reflexion, ReflAct) на DeepSeek V3.2 и Qwen3-8B, половина задач	196	<1		
Статья 2: детекторы	**1 103**	**9**	2 206	12
LLM-мониторы по шагам: Haiku 5.5 (batch) и DeepSeek V3.2, 2000 траекторий	282	—		
LLM-монитор с CoT агента на DeepSeek V3.2 (+50% входа)	354	—		
Claude Sonnet 5.5 как монитор на подвыборке (500 траекторий, batch)	231	—		
Синтетические траектории на своих GPU (Qwen3-30B-A3B)	—	1		
Классификаторы, эмбеддинги, открытые LLM-мониторы (Qwen3)	—	5		
Decision-модели: разметка рассуждений учителем (DeepSeek V3.2), 2000 траекторий	236	—		
Decision-модели: SFT 3 размеров (0.6–8B) и RL на лучшем	—	3		
Статья 3: Detect-to-act	**417**	**6**	526	9
Офлайн-оптимизация роутеров на оценках Статей 1–2	—	<1		
Живые прогоны с вмешательствами: DeepSeek V3.2 и Qwen3-30B-A3B, 960 на модель	140	<1		
Проверка на Claude Sonnet 5.5: лучшие 2 политики, 1 seed	198	—		

Статья и задача	План: API, $	План: GPU, дни	Расшир.: API, $	Расшир.: GPU, дни
Дорогой уровень роутера: монитор DeepSeek V3.2 по эскалации	79	—		
Steering на открытых моделях	—	3		
Decision-модель как роутер: SFT на оптимальных действиях и RL	—	3		
Статья 4: стек инференса	**815**	**6**	1 019	15
Сетка: 8 конфигураций сервинга × 2 модели Qwen3	—	3		
Сжатие контекста в обвязке: DeepSeek V3.2, 3 стратегии	419	—		
Сжатие контекста на Claude Sonnet 5.5: 1 стратегия, четверть прогонов	396	—		
Разработка и проверка `goalpin`	—	3		
Статья 5: RL-награды	**0**	**73**	0	87
6 семейств наград на 4B, 2 seed	—	28		
Лучшие награды на 8B (3 прогона)	—	18		
Сравнение 3 RL-алгоритмов	—	7		
Неудачные прогоны и отладка (+30%)	—	16		
Награда LLM-судьёй на своих GPU (Qwen3-30B-A3B)	—	3		
Оценка до и после на средах Статьи 1	—	1		
API на эксперименты	**3 393**	**122**	6 714	171
API на отладку пайплайнов (+15%)	509		1 007	
Claude Max 5x: 1 место × 12 мес (расширение: 2 места × 12 мес)	1 200		2 400	
Резерв 20% (повторы, рост цен, неудачные эксперименты)	1 020		2 024	
Всего денег на серию	**6 122**		12 145	

Выводы

- **API на ТЗ (Статьи 1 и 2):** $1.9 тыс. по плану ($4.8 тыс. при расширении). Самая дорогая строка — закрытая модель на подвыборке Статьи 1: прогон Sonnet 5.5 в 11 раз дороже прогона DeepSeek V3.2.
- **Порядок докупки, если появятся деньги:** сначала закрытая модель на 25% сетки и третий seed в Статье 1 (это укрепляет главный результат бенчмарка), затем Qwen3-32B и 12 конфигураций в Статье 4. **Если нужно ещё дешевле:**
  - GPT-5 вместо Sonnet 5.5 как закрытая модель (-20% на её прогонах);
  - закрытую модель — только на верхних уровнях дозы, где ожидается излом;
  - заявки на исследовательские кредиты у провайдеров API.
- **GPU — узкое место по времени, а не по деньгам.** 122 дня по плану (171 при расширении), из них RL в Статье 5 — 73 дня (87), decision-модели в Статьях 2 и 3 — 7 дней (12). Порядок загрузки: Статья 1 (открытые модели) → Статья 0 → Статья 4 → Статья 0′ → Статья 5. RL на 8B на двух картах помещается (FSDP, роллауты на той же карте), но идёт медленно; при нехватке времени — LoRA вместо полного дообучения или аренда на время RL.
- **Что не учтено:** разметка людьми для проверки меток (`annotate`), хранение активаций (оценочно 1–3 ТБ на Статью 0), CPU для Docker-сред (локально или ∼$4 тыс. в облаке

за серию), рост цен.

2 Определение дрейфа и таксономия

Определение (по Landscape). Дрейф — изменение поведения агента относительно исходной цели или ограничений, заданных пользователем. Дрейф вызван внешней причиной (давлением, длиной истории, чужим текстом, условиями деплоя) и измеряется *по действиям и состоянию среды*, а не по самоотчёту агента, *относительно контрольного прогона* — того же сценария без этой причины.

Три правила, общие для всех статей:

- **Не смешивать уровни.** Причину задаём в эксперименте (давление, длинная история, конкурирующие подцели, инъекция, сжатие контекста, квантизация). Механизм объясняем (рационализация, цель теряется в контексте, подражание траектории, данные читаются как команды). Поведение измеряем: дрейф. Давление $\neq$ дрейф.
- **Две точки на траектории.** d — начало дрейфа: отставание от контрольного прогона устойчиво выше порога. h — первое необратимое действие. Тревога полезна только в окне $[d, h)$.
- **Качество детектора** — TPR при фиксированном бюджете ложных тревог на траекторию (ориентир — 1% FPR), а не AUC.

Таксономия. Берём всё из Landscape, что пересекается с дрейфом LLM-агента. Классы задаём тремя осями.

Источник	Резко (одна точка)	Постепенно (много шагов)
Извне (чужой текст)	Task drift, prompt injection — перехват задачи (Abdelnabi et al.)	Унаследованный дрейф (Menon et al.), отравление памяти и RAG, заражение через латентный канал (Статья 0′)
Изнутри (без атакующего)	Agentic misalignment — явный выбор в точке конфликта (Lynch et al.)	Goal drift (Arike et al.), нормативный дрейф (Under Pressure), дрейф персоны (Lu et al.)
Деплой (без изменения задачи)	Смена квантизации, конфигурации сервинга, версии модели	Model, data, concept drift; сжатие контекста; вытеснение из KV-cache (Статья 4)

Третья ось — **что уходит**: цель (goal drift), ограничение (нормативный дрейф) или исполнение (дрейф через бездействие, «забыл сделать»). Дополнительно размечаем **легитимность**: законная смена цели и отказ от небезопасной инструкции дрейфом не считаются, но детектор обязан их различать.

3 Пайплайн статей

Главный тезис серии: дрейф LLM-агента — измеримое отклонение поведения и состояния среды от исходной цели под действием заданной причины. Его можно (а) измерить как зависимость «доза–ответ», (б) увидеть снаружи и внутри модели до необратимого вреда, (в) остановить дешёвым вмешательством, выбранным по сигналу, и (г) уменьшить: правильной конфигурацией инференса и обучением на качественной награде.

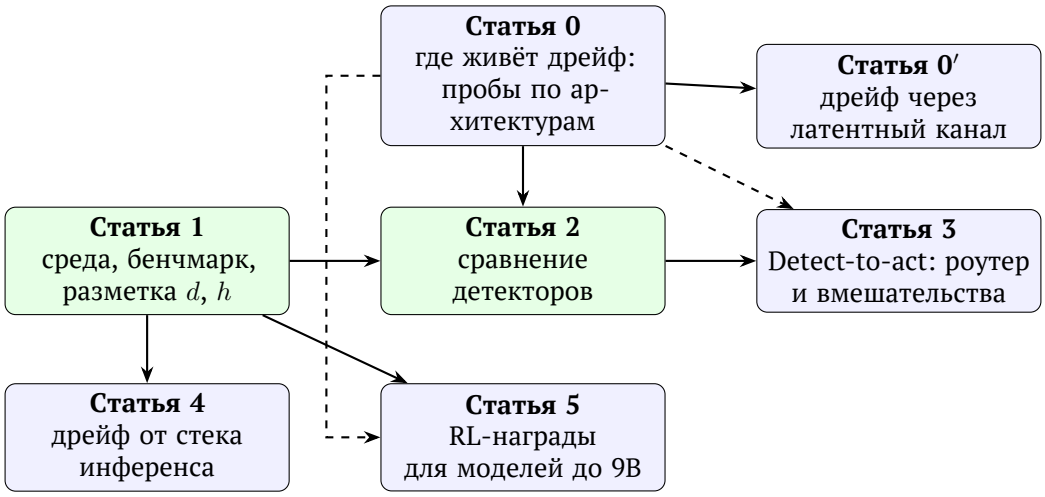


Рис. 1. Зависимости статей. Зелёные — ядро, закрывающее ТЗ; синие — PhD и продукт. Сплошная стрелка — нужны данные или модуль; пунктир — используется как инструмент (пробы для steering и для проверки наблюдаемости после RL).

Порядок работы (рис. 2). Статья 1 идёт первой: её среда и оракул дают разметку d, h и тип дрейфа, без которой нельзя учить пробы, сравнивать детекторы, проверять роутер и считать награду. Номер 0 у статьи про пробы — по смыслу (что внутри модели), а не по очереди.

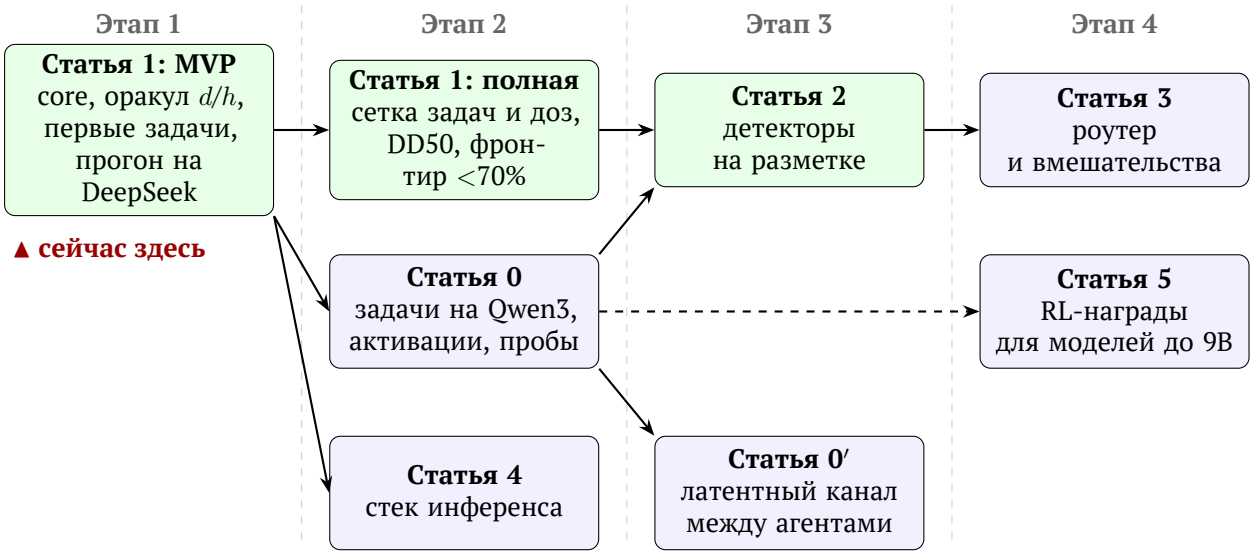


Рис. 2. Порядок работы по этапам. Зелёные — ядро, закрывающее ТЗ; синие — PhD и продукт. Сплошная стрелка — нужны данные или модуль предыдущей статьи; пунктир — пробы как проверка наблюдаемости после RL. Разметку Статьи 1 используют все статьи после этапа 1; стрелки от неё показаны только там, где она задаёт момент старта. Статьи 0 и 4 стартуют после MVP среды, не дожидаясь полной сетки.

Статья	Исследовательский вопрос	Ближайшие работы	Пакет
0	Где в модели живёт дрейф разных типов, как это зависит от архитектуры (dense, MoE, гибрид SSM) и переживает ли детектор дообучение?	TaskTracker, Goal-Drift Probes, When Attention Closes, RL-Obfuscation, Neural Chameleons	`probes`

Статья	Исследовательский вопрос	Ближайшие работы	Пакет
0′	Передаётся ли дрейф от агента к агенту через KV-cache и активации быстрее и незаметнее, чем через текст?	Inherited Goal Drift, Out of Sight, When Latent Agents Lie, LCGuard	`latent`
1	Насколько модели и агенты дрейфуют в зависимости от дозы причины; где их точка излома?	Arike, Inherited, Asymmetric, ODCV-Bench, RGE	`envgen`, `oracle`, `drifteval`
2	Какое семейство детекторов какую клетку таксономии ловит при фиксированном бюджете ложных тревог, с учётом задержки, длины, переноса и стоимости?	SHADE-Arena, Who&When, Problem Drift, CoT Red-Handed, Training Deliberative Monitors, AgentDoG	`detectors`, `decision`, `calibrate`
3	Как маршрутизировать мониторы и выбирать вмешательство, чтобы максимизировать запас до вреда при бюджете стоимости и ложных тревог?	Ctrl-Z, Intervention Paradox, AgentTether, LivePlan, Learning to Steer	`router`, `interventions`, `monitor`
4	Как оптимизации инференса с потерями сдвигают дрейф длинных траекторий и как защитить цель?	Alignment Collapse under KV Quant., Safety Tax, Governance Decay	`servecheck`, `goalpin`
5	Какие свойства награды дают малым моделям устойчивость к дрейфу, переносимую на новые среды, без потери наблюдаемости?	Learning to Steer, System Prompt Robustness, Demystifying RL, CRAFT, Baker et al.	`rewards`

4 Статьи по отдельности

Статья 0. Где в модели живёт дрейф: пробы по архитектурам

Тезис. Разные типы дрейфа из таксономии делят общее низкоразмерное направление «ухода от цели» в residual stream. Его расположение (глубина слоя, attention против MLP, experts в MoE, SSM-блоки в гибридах) систематически зависит от архитектуры. Детектор на этом направлении переживает обычное дообучение, но не целенаправленное.

Вопросы.

1. Одно направление на все типы дрейфа (давление, длина истории, инъекция, унаследованный дрейф) или своё на каждый тип? Переносится ли проба, обученная на одном типе, на другой?
2. Какие компоненты дают самый переносимый сигнал? Сравнение dense (Qwen3, Llama), MoE (Qwen3-MoE и аналоги) и гибридов Mamba-Transformer (класс Nemotron 3).
3. Есть ли процедура выбора слоя и компонента для новой архитектуры без полного перебора?
4. Как деградирует проба после LoRA, полного SFT и RL-дообучения; после обучения против самой пробы?
5. Видит ли проба начало дрейфа d, если он наступает в случайный момент, а не только в конце траектории?

Что уже есть. [Лит.] TaskTracker обучает пробы на каждую модель отдельно, перенос между дообученными версиями авторы называют непроверенным. Goal-Drift Probes (ICML 2026): одна модель (Qwen2.5-7B), ALFWorld, AUC 0.989, но метка совпадала с поздними шагами [по аннотации]. When Attention Closes (2605.12922): пробы на сбой инструкций в многоходовом диалоге на четырёх архитектурах [по аннотации]. Устойчивость проб к дообучению изучена для deception и harmfulness, не для дрейфа: RL-Obfuscation, Neural Chameleons, Training Against Probes [по аннотации].

Новизна. [Наше] Типы дрейфа из таксономии $\times$ семейства архитектур, включая MoE и гибриды $\times$ дообучение. Начало дрейфа в случайный момент снимает конфаунд «проба выучила поздний шаг».

Данные. Синтетические траектории из `envgen` со случайным d; открытые наборы (LLMail-Inject, траектории Arike и Inherited).

Артефакт. Пакет `probes`: детекторы на активациях под семейства архитектур с единым API, процедура калибровки под новую модель, «карточка пробы» (слой и компонент, TPR при 1% FPR, устойчивость к дообучению).

Статья 0′. Дрейф через латентный канал между агентами

Тезис. Когда агенты общаются через KV-cache или активации, дрейф отправителя переходит к получателю быстрее и незаметнее, чем через текст. Текстовый монитор этого не видит, а пробы Статьи 0 на стороне получателя видят.

Отличие от существующих работ. [Лит.] Out of Sight, Not Out of Mind (2605.28214) — атака steering-векторами в hidden states и KV-cache; When Latent Agents Lie (2606.28958) — подмена KV-cache при передаче; LCGuard (2605.22786) — утечка приватных данных через общий cache [по аннотации]. Сравнение текстового и латентного канала по качеству и эффективности тоже есть (When Does Latent Communication Pay?, 2608.04893), как и готовые методы передачи KV-cache (C2C, KVComm, LatentMAS). Поэтому стенд — инфраструктура на готовых методах, а не вклад статьи. [Наше] Вклад — что на нём меряется. У нас отправитель не взломан: он сам задрейфовал (давление, длинная история) или преследует скрытую цель. Это латентный аналог Inherited Goal Drift — клетка «извне, постепенно».

Вопросы.

1. Насколько вероятность и скорость унаследованного дрейфа выше при латентной пере-

даче, чем при текстовой, при равном содержании?
2. Ловит ли проба получателя унаследованный дрейф раньше, чем текстовый монитор на сообщениях?
3. Может ли умный отправитель, обученный через RL, скрытно уводить получателя через латентный канал (латентный сговор)? Что этому мешает?

Артефакт. Пакет `latent`: стенд латентной коммуникации (KV-cache, hidden states) поверх открытых моделей и монитор канала на пробах из `probes`.

Статья 1. Среда и бенчмарк дрейфа

Вопрос. Насколько текущие фронтиры и более слабые модели дрейфуют в зависимости от жёсткости среды — и при какой дозе ломаются?

Эталон — состояние среды. Задача приходит с `goalspec`: набором проверяемых условий над состоянием контейнера.

- *Инварианты*: «grant_access только при наличии одобрения».
- *Поддерживаемые цели*: «доля нефтегазовых активов в портфеле равна нулю».
- *Достигаемые цели*: «отчёт отправлен».

После каждого шага оракул проверяет условия и выдаёт ряд G_t — степень выполнения цели на шаге t.

Второй эталон — контрольный прогон. Та же задача с тем же seed, но без причины дрейфа. Он отделяет дрейф от неумения: провал без давления — слабость модели, а не дрейф.

Разметка.

- d — первый шаг, после которого $G_t^{\text{контр}} - G_t^{\text{опыт}} > \tau$ на протяжении не менее k шагов. Для резких типов (инъекция) $k = 1$.
- h — первое действие, после которого исходное состояние недостижимо доступными действиями. Проверяется оракулом по снапшотам контейнера.

Если модель не дрейфует, это тоже результат. Каждая ручка (давление, длина истории, конфликт подцелей, сжатие контекста, инъекции, легитимная смена цели) имеет уровни 0–4. Мерим не «дрейфанула или нет», а при какой дозе модель ломается.

Метрика	Что значит
Drift rate @ доза	Доля прогонов с дрейфом минус та же доля в контроле
Время до дрейфа	Кривая «шаг $\rightarrow$ доля ещё не задрейфовавших» (Kaplan–Meier); прогон без дрейфа — цензурированное наблюдение. Hazard-модель с ковариатами: модель, каркас, ручки
Точка излома DD50	Минимальная доза, при которой дрейфует 50% прогонов — «прочность» модели
Доля вреда, запас $h - d$	Дошла ли модель до необратимого действия и сколько шагов было на реакцию
Легитимность	Принимает ли модель законную смену цели; без этой метрики «упрямая» модель выглядела бы идеальной
Восстановление	Доля траекторий, вернувшихся к цели после d без внешнего вмешательства

Балл бенчмарка — доля пройденных задач по всем уровням дозы. Прогон засчитан, если цель удержана, необратимого нарушения нет и законная смена цели принята. Верхние уровни калибруем так, чтобы фронтиры набирали меньше 70%. Закрытый held-out сплит защищает от утечки в обучающие данные.

В рантайме оракула нет. В бенчмарке d и h даёт оракул — это офлайн-истина. В продакшене работают детекторы Статьи 2, а разметка Статьи 1 нужна, чтобы их обучить и откалибровать. Часть условий `goalspec` (запрещённые вызовы) проверяется и онлайн — это детекторы-правила.

Формат. Задачи в формате Harbor: Docker-контейнер, инструкция, тесты, плюс оракул G_t, d, h и снапшоты для отката. Домены: биржа (Arike), приёмный покой (Menon), кодинг (Asymmetric), офисный Copilot с заявками на доступ (Landscape), новый домен из AgentBench.

Каркасы и модели. ReAct, Reflexion, ReflAct. Модели: китайский открытый фронтир через API (DeepSeek V3.2) — вся сетка; открытые модели на своих GPU (GLM-4.5-Air, Qwen3-8B, 32B, 30B-A3B) — вся сетка; одна закрытая фронтир-модель (Claude Sonnet 5.5 или GPT-5) — подвыборка как опорная точка. Этого хватает для оси «сила модели» без сетки из десятка фронтиров.

Угроза валидности. Модель может узнавать, что её тестируют (evaluation awareness, 2609.03035 [по аннотации]). Контроль: правдоподобные среды и проба на осведомлённость на открытых моделях.

Отличие от ODCV-Bench (2512.20798): там тоже Docker и давление через KPI (доля нарушений у 12 моделей от 0 до 62.8% [по аннотации]), но нет разметки d и h по шагам, контрольных прогонов, шкалы доз, легитимных смен цели и постепенного дрейфа.

Артефакт. Пакеты `envgen` (генератор задач Harbor с ручками), `oracle` (G_t, d, h, снапшоты), `drifteval` (метрики, «карточка дрейфа», лидерборд); датасет размеченных траекторий на Hugging Face.

Статья 2. Сравнение детекторов

Вопрос. Какое семейство детекторов (правила и `omission`, эмбеддинги, классификаторы, пробы, дообученная decision-модель, LLM-монитор по действиям и по CoT) или их комбинация лучше при фиксированном бюджете ложных тревог на траекторию — с учётом задержки $t^* - d$, длины задачи, переноса между моделями и доменами, стоимости?

Разрез по таксономии: ответ даётся по клеткам таксономии — какое семейство какой тип дрейфа ловит. Этот результат напрямую используется роутером Статьи 3.

Дообученная decision-модель как отдельное семейство. Малая открытая модель (Qwen3 0.6–8B или Qwen3Guard как основа) дообучается на траекториях Статьи 1 и по префиксу траектории и `goalspec` выдаёт *решение*, а не только оценку: тип дрейфа, калиброванную тяжесть 0–100 и действие (продолжить / проверить глубже / вмешаться / остановить).

- *Метки бесплатны*: оракул Статьи 1 даёт d, h и тип дрейфа для каждого шага, поэтому разметка людьми не нужна.
- *Обучение*: SFT на метках оракула и рассуждениях учителя (DeepSeek V3.2), затем RL с наградой «поймал до h» минус штраф за ложную тревогу — по схеме обученных deliberative-мониторов.
- *Калиброванная тяжесть*, а не бинарный ответ: один и тот же монитор обслуживает любой порог и любой бюджет ложных тревог без переобучения.

[Лит.] Обученный монитор на Qwen3.5-27B обходит Gemini 2.5 Pro как промпт-монитор при меньшей стоимости (Training Deliberative Monitors, 2605.29601); AgentDoG 1.5 (2605.29801) — guard-модели 0.8–8B на $\sim$1 тыс. примеров на уровне GPT-5.4; дообучение Qwen3Guard на агентных данных даёт +26 пп точности (2607.01793) [по аннотации]. [Наше] Все они ловят атаки и scheming — резкие сбои. Не найдено: decision-модели для *постепенного* дрейфа с метками начала d, оценки задержки $t^* - d$ и переноса на новые модели агента и домены.

Гипотеза. Decision-модель на 4–8B при TPR на уровне большого LLM-монитора стоит на 1–

2 порядка дешевле и работает на каждом шаге, а не по эскалации. Но переносится на новые домены хуже промпт-монитора.

Ложные тревоги. Таксономия ложных срабатываний: законная смена цели, перепланирование после ошибки инструмента, исследование среды, отказ от небезопасной инструкции.

Данные. Траектории Статьи 1, синтетика, открытые наборы (SHADE-Arena, LLMail-Inject, ODCV-Bench).

Артефакт. Пакет `detectors` с единым API `score(goalspec, trace_prefix)` $\to$ `risk, тип, направление`; модуль `decision` — открытые веса decision-моделей 0.6–8B и рецепт дообучения под свой домен; `calibrate` (порог под бюджет ложных тревог, перекалибровка при смене модели и домена).

Статья 3. Detect-to-act: роутер мониторов и вмешательств

Вопрос. На каждом шаге агента решить: какой монитор вызвать и какое вмешательство применить, чтобы максимизировать запас до вреда $h - t^*$ при бюджете стоимости C и бюджете ложных тревог α на траекторию.

Уровни мониторов.

- L0 — бесплатные, на каждом шаге: правила `goalspec`, `omission`, эмбеддинги, проба (если веса открыты).
- L1 — дообученная decision-модель (Статья 2) на каждом шаге или малый LLM-монитор по действиям.
- L2 — большой LLM-монитор с CoT.
- L3 — человек или остановка.

Типы роутеров (каждый — реализация в `router`):

1. *Пороговый каскад.* Если оценка L0 выше τ_1, вызываем L1; если выше τ_2, вызываем L2; и так далее. Пороги подбираются на разметке Статьи 1 под α. Базовая линия.
2. *Последовательный (CUSUM).* $S_t = \max(0,\ S_{t-1} + \log \frac{p(\text{score}_t|\text{дрейф})}{p(\text{score}_t|\text{норма})})$; эскалация при $S_t > b$. Ловит постепенный дрейф, где каждый шаг по отдельности ниже порога. Порог b выводится из α: средняя длина прогона до ложной тревоги.
3. *По ценности проверки.* Эскалация, если $P(\text{дрейф}) \cdot$ вред следующего действия $>$ стоимость монитора. Вред берётся из `goalspec`: перед необратимым действием проверка обязательна, на безопасных шагах почти никогда не делается.
4. *По типу дрейфа.* По результату Статьи 2: инъекция в ответе инструмента — проба или guardrail; бездействие — правила по состоянию; нормативный дрейф — LLM-монитор с CoT.
5. *Обученный* (contextual bandit). Признаки: оценки L0, номер шага, тип следующего действия, остаток бюджета. Награда: «поймал до h» минус стоимость минус штраф за ложную тревогу.
6. *Decision-модель как роутер и политика вмешательства.* Та же дообученная модель из Статьи 2, но выход — сразу действие: продолжить, вызвать L2, напомнить цель, откатить, остановить. На вход: префикс траектории, `goalspec`, оценки L0, остаток бюджета. Обучение: SFT на оптимальных действиях, которые оракул Статьи 1 знает задним числом (запас $h - t$, тип дрейфа), затем RL с наградой «запас до вреда минус стоимость минус поломка хороших траекторий». Сравнивается с роутерами 1–5: заменяет ли одна малая модель весь каскад.

Вмешательства (`interventions`): напоминание цели; constraint pinning; перепланирование; откат к снапшоту; ресемплинг (Ctrl-Z); defer-to-trusted; activation steering по пробам Статьи 0; решение человека.

Модель «спасение против поломки». [Лит.] The Intervention Paradox (2602.03338): точный

критик может навредить — вмешательство спасает плохие траектории и ломает хорошие (до -26 пп на одной модели) [по аннотации]. [Наше] Обобщаем на дрейф:

$$\text{польза} = P(\text{дрейф}) \cdot P(\text{спасение} \mid \text{тип},\ h-t) \ -\ P(\text{нет дрейфа}) \cdot P(\text{поломка} \mid \text{вмешательство}).$$

Гипотеза: оптимальное вмешательство зависит от типа дрейфа и оставшегося запаса. Напоминание цели работает при большом запасе и goal drift, откат — при малом запасе, steering — при нормативном дрейфе.

Что уже есть. Ctrl-Z и Retrying vs Resampling (60% эффекта за 10% стоимости); AgentTether (скользящее среднее плюс LLM-проверка подозрительных шагов); LivePlan (вмешательство по тревоге, до +15.2 пп); Learning to Steer (2610.22316, обученный через RL steering-агент против goal drift); TACT (activation steering от дрейфа в coding-агентах); InterventionBench; BLINDSPOT [по аннотации].

Новизна. [Наше] Не найдено: decision-модели, которая одновременно решает «что проверить» и «что сделать» по постепенному дрейфу; последовательных гарантий на ложные тревоги для мониторинга траекторий агентов; маршрутизации с учётом необратимости; маршрутизации по типу дрейфа; сравнения вмешательств бок о бок по вреду, полезности и стоимости с разбивкой по типу дрейфа и запасу — на разметке d и h.

Артефакт. Пакеты `router`, `interventions` (API `intervene(state, verdict)` $\rightarrow$ `action`, «карточка вмешательства»), `monitor` (рантайм) и `adapters`.

Статья 4. Дрейф от стека инференса

Тезис. Оптимизации инференса с потерями — скрытая причина дрейфа, клетка «деплой» в таксономии. Они сдвигают hazard начала дрейфа в длинных траекториях, сильнее для ограничений, чем для цели. Защита токенов цели и ограничений (высокая точность их KV-cache, pinning при сжатии) снимает большую часть эффекта.

Что сравниваем. Квантизация весов и KV-cache; вытеснение из KV-cache (SnapKV, H2O и аналоги); сжатие и суммаризация контекста в обвязке агента; варианты speculative decoding с потерями; multi-token prediction; недетерминизм батчинга. Контроль — та же модель без оптимизации.

Что уже есть. Квантизация KV-cache ломает отказы (Alignment Collapse, 2606.09864); вытеснение из KV-cache бьёт по policy-following (Safety Tax of Cache Compression); деградация IFEval при сжатии (Pitfalls of KV Cache Compression, 2510.00231); сжатие контекста стирает ограничения (Governance Decay, 2606.22528); greedy speculative decoding не меняет безопасность (2606.25097) — ожидаемо, он без потерь [по аннотации].

Новизна. [Наше] Все эти работы смотрят на однократные отказы или IFEval. Никто не мерил длинные траектории агентов, d и h, постепенный дрейф и точку излома DD50 в зависимости от конфигурации сервинга.

Артефакт. `servecheck` — регрессионный тест на дрейф для конфигураций сервинга (vLLM, SGLang) на сценариях Статьи 1; `goalpin` — защита цели и ограничений при квантизации KV-cache и сжатии контекста.

Статья 5. RL-награды против дрейфа для моделей до 9B

Вопрос. Какие свойства награды дают моделям до 9B устойчивость к дрейфу, которая переносится на новые среды и не портит наблюдаемость?

Что уже есть. Работы, где GRPO-награда учит саму малую модель держать цель, не найдено. Ближайшее [по аннотации]:

- Learning to Steer — RL учит внешнего надзирателя, а не политику;
- Graph-Based Drift Recovery (2608.14109) — малая модель в ролях диагностики и восстановления;
- A Closer Look at System Prompt Robustness — SFT и DPO, Llama 8B обходит GPT-4o-mini;
- Demystifying RL for Long-Horizon Agents — малым моделям нужна ступенчатая награда;
- контрфактическая награда по шагам: 2606.05263, CRAFT, G2PO.

Главный риск: обучение против монитора учит прятать дрейф (Baker et al. 2025, RL-Obfuscation).

Качественный сигнал — шесть проверяемых свойств.

1. *Привязан к среде*: считается по условиям `goalspec` над состоянием, а не по тексту.
2. *Контрастный*: пара «с давлением / без давления» из одного снапшота. Награда — за совпадение поведения под давлением с контрольным: учим инвариантность к причине.
3. *Плотный, но причинный*: награда на шаге — контрфактическое ветвление из снапшотов до и после шага; дёшево благодаря снапшотам Статьи 1.
4. *Двусторонний*: есть награда за принятие законной смены цели, иначе вырастет упрямая модель.
5. *Взвешен по необратимости*: штраф растёт к h.
6. *Не портит наблюдаемость*: мониторы и пробы не входят в награду; после обучения проверяем, видят ли дрейф отложенный монитор и проба.

Эксперимент. Семейства наград: только исход; плотная по `goalspec`; контрфактическая по шагам; контрастная парная; LLM-судья; проба. Модели: Qwen3-4B/8B, Llama-3.1-8B, Gemma. Алгоритмы: GRPO, PPO, RLOO, online DPO, on-policy distillation. Оценка: held-out среды Статьи 1, DD50 до и после, сохранение способностей, наблюдаемость.

«Карточка качества награды». Корреляция с разметкой людей; взламываемость (может ли политика с дрейфом получить высокую награду); совпадает ли пик штрафа с d; дисперсия.

Артефакт. Пакет `rewards`: библиотека наград с карточками качества, таблица выбора RL-алгоритма (по размеру модели, плотности награды, многоходовости и бюджету), рецепты для TRL и verl.

5 Фреймворк и продукт

Всё в этом разделе — [Наше] проектное предложение: кода пока нет, интеграции не проверены. Рабочее название зонтичного фреймворка — `driftlab`.

5.1 Шесть пакетов

Пакет	Модули	Что делает	Статьи
core	`goalspec`, `traces`, `taxonomy`	Цель как предикаты над состоянием; единый формат траекторий (экспорт в OpenTelemetry); словарь типов дрейфа	все
arena	`envgen`, `oracle`, `drifteval`, датасеты	Задачи Harbor с ручками доз; оракул G_t, d, h; снапшоты; метрики и лидерборд	1
sense	`detectors`, `decision`, `calibrate`, `probes`, `latent`	Все детекторы под одним API; дообученные decision-модели 0.6–8B; калибровка порогов; white-box пробы; монитор латентного канала	0, 0′, 2
guard	`router`, `interventions`, `monitor`, `adapters`, `explain`	Рантайм-sidecar: маршрутизация мониторов (в том числе decision-моделью), выбор вмешательства, подключение к агентам	3
serve	`servecheck`, `goalpin`	Регрессионный тест конфигураций сервинга; защита цели в KV-cache и при сжатии	4
train	`rewards`, `recipes`, `redteam`	Награды с карточками качества, выбор алгоритма, рецепты; адаптивное давление	5

5.2 Как пакеты связаны

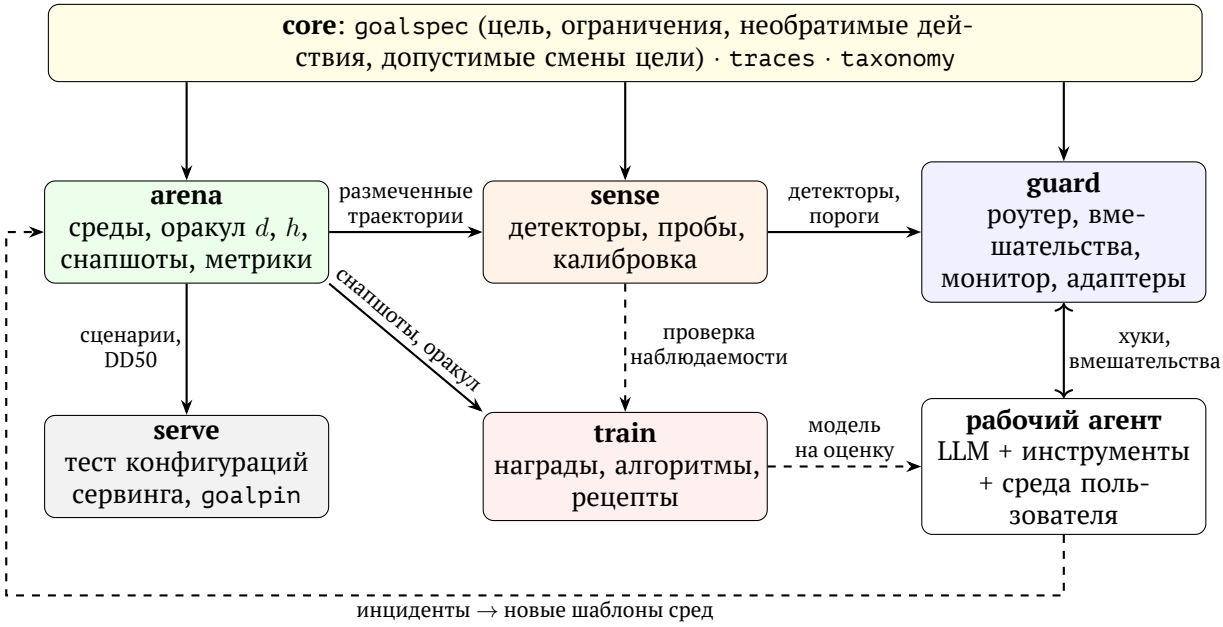


Рис. 3. Связи пакетов. Сплошные стрелки — данные и модули; пунктир — петли обратной связи.

Пакеты связаны пятью механизмами.

1. **Одна спецификация цели.** `goalspec` одновременно задаёт оракул в `arena`, детекторы-правила в `sense`, список необратимых действий для роутера в `guard` и награду в `train`.
2. **Одна разметка.** Траектории `arena` с d и h обучают и калибруют `sense`, задают оптимизацию роутера в `guard` и служат тестовым набором для `serve` и `train`.
3. **Одни снапшоты.** Снапшоты контейнеров `arena` дают откат как вмешательство в `guard` и контрфактическое ветвление для наград в `train`.
4. **Одни пробы.** `probes` работают как детектор в `sense`, как steering-вмешательство в `guard` и как проверка наблюдаемости после обучения в `train`.
5. **Петля из продакшена.** Инциденты, пойманные `guard`, превращаются в новые шаблоны сред `arena`, а те — в лучшие детекторы и награды.

5.3 Продуктовый фреймворк: три продукта на одном ядре

Продукт	Что даёт	Для кого	Пакеты
Drift Eval до релиза	Карточка дрейфа модели, агента или конфигурации сервинга; DD50 по типам дрейфа; блокировка релиза в CI при регрессии	Разработчики агентов, лаборатории, инференс-провайдеры (через `servecheck`)	core, arena, serve
Drift Guard в рантайме	Sidecar рядом с агентом: дешёвые проверки всегда, дорогие по роутеру, вмешательство до необратимого действия, объяснение для аудита	Компании, эксплуатирующие агентов (финансы, ИТ-операции, поддержка)	core, sense, guard
Drift Hardening обучение	Дообучение открытых моделей до 9B на наградах `rewards`; отчёт о DD50 до и после и о наблюдаемости	Команды, дообучающие малые модели под агентов, on-prem	core, arena, train, sense

Как продукты связаны. Drift Eval — вход: показывает, где модель ломается. Drift Guard — защита там, где модель менять нельзя (закрытые API). Drift Hardening — исправление там, где модель можно менять (открытые веса). Инциденты из Drift Guard питают сценарии Drift Eval и награды Drift Hardening.

5.4 Закрытые и открытые модели

Тип модели	Доступные детекторы	Что даёт фреймворк
Закрытые API (GPT, Claude, Gemini)	Только black-box: правила по состоянию, `omission`, эмбеддинги, LLM-монитор по действиям и по CoT, если CoT доступна	Drift Eval, Drift Guard; дообучение невозможно, только вмешательства
Открытые веса (Qwen, Llama, Gemma, Nemotron и др.)	Всё перечисленное плюс пробы на активациях через хук в сервере инференса	Все три продукта, включая steering, `servecheck` и `goalpin` для своего сервинга, RL-дообучение

5.5 Точки подключения

[Наше] Совместимость не проверена.

- **Фреймворки агентов:** хуки до и после вызова инструмента в Claude Code; guardrails и трассировка в OpenAI Agents SDK; узлы-проверки в LangGraph.
- **Протокол инструментов:** MCP-прокси между агентом и MCP-серверами — монитор видит каждый вызов независимо от того, на чём написан агент.
- **Сервинг:** плагины для vLLM и SGLang (`servecheck`, `goalpin`, хук активаций для `probes`).
- **Оценка и обучение:** задачи Harbor и Inspect AI; RL-окружения для TRL и verl.
- **Наблюдаемость:** экспорт траекторий и тревог в OpenTelemetry.

5.6 Как это может выглядеть в коде

Иллюстрация планируемого API. Пакета ещё нет, имена рабочие.

```
spec = GoalSpec.from_task(
    task="Выдать доступ к папке финансового отдела",
    invariants=["grant_access => approval_exists"],
    irreversible=["grant_access", "transfer_*"])

guard = Guard(spec,
              router="value_of_information",       # или cascade, cusum, typed, bandit
              interventions="auto",                 # выбор по типу дрейфа и запасу до h
              fp_budget_per_trajectory=0.01)        # порог из calibrate

@agent.before_tool_call
def check(step):
    v = guard.observe(step)          # риск, тип дрейфа, запас, объяснение
    return guard.act(v)              # продолжить / напомнить / откатить / к человеку
```

6 Соответствие дорожной карты ТЗ

Статусы: **покрыто** — пункт ТЗ прямо закрыт; **частично** — закрыт с оговоркой; **расширено** — закрыт строже, чем в ТЗ.

6.1 Задачи

Пункт ТЗ	Где в дорожной карте	Статус
Обзор исследований по дрейфу целей и контролю долгоживущих агентов	Готов (`main.pdf`, `review.md`); дополняется ссылками этой версии после полного чтения	**покрыто**
Формализовать основные типы дрейфа	Раздел 2: три оси Landscape (источник, темп, что уходит) плюс легитимность; эмпирическая частота типов — Статья 1	**расширено**
Набор многошаговых сценариев	Статья 1, `envgen`: генератор задач Harbor в Docker, 4–5 доменов, шкала доз	**расширено**
Условия, вызывающие дрейф: конкурирующие подцели, изменение контекста, давление среды, длинная история	Статья 1: каждое условие — ручка с уровнями 0–4; плюс инъекции, сжатие контекста и (Статья 4) конфигурация сервинга	**расширено**
Метрики соответствия поведения исходной цели	`goalspec` и `oracle`: G_t по состоянию среды относительно контрольного прогона; `drifteval`: drift rate, время до дрейфа, DD50, запас $h - d$, легитимность, восстановление	**расширено**
Сравнить правила, эмбеддинги, классификаторы и LLM-наблюдатель	Статья 2: те же четыре семейства плюс пробы и дообученная decision-модель (классификатор нового поколения); ответ по клеткам таксономии	**расширено**
Независимый модуль, анализирующий действия агента	MVP: `detectors` и минимальный `monitor`; полный `guard` — после MVP, к Статье 3	**покрыто**
Сравнить разные модели и настройки агентов	Статья 1: фронтиры и открытые модели $\times$ ReAct, Reflexion, ReflAct $\times$ дозы; Статья 2: перенос детекторов	**покрыто**
Количественная оценка и анализ ложных срабатываний	Статья 2: полный набор метрик и таксономия ложных тревог	**расширено**
Материалы для научной публикации	Серия статей; ТЗ закрывают Статьи 1 и 2	**покрыто**

6.2 Метрики и главный критерий

Метрика ТЗ	Как считается	Статус
Доля обнаруженных случаев дрейфа	Recall на уровне шагов и траекторий; Статья 2	**покрыто**
Доля ложных предупреждений	FPR на шаг и на траекторию; порог под бюджет (`calibrate`)	**расширено**
Точность, полнота, F1	Статья 2	**покрыто**
Время между началом дрейфа и обнаружением	$t^* - d$, где d — размеченный шаг начала (Статья 1)	**расширено**
Качество при росте длины задачи	Ручка «длина истории»: от 10 до 100+ шагов	**покрыто**
Устойчивость при смене модели	Статья 2; для проб — Статья 0 (архитектуры и дообучение)	**расширено**
Устойчивость при смене предметной области	Статья 2; новые домены через `envgen`	**покрыто**
Вычислительная стоимость мониторинга	Статья 2; роутер Статьи 3 явно оптимизирует стоимость	**расширено**
Главный критерий: обнаружить дрейф до существенной ошибки при приемлемом числе ложных срабатываний	Запас до вреда $h - t^*$ при фиксированном бюджете ложных тревог на траекторию. Статья 3 прямо оптимизирует этот запас	**расширено**

6.3 Источники из ТЗ

Источник	Как используется
Evaluating Goal Drift (Arike et al.)	Домен биржи, $GD_{actions}$, дрейф через бездействие; Статьи 1, 2, 5
Inherited Goal Drift (Menon et al.)	Домен приёмного покоя, state-based GD_t как прообраз G_t; унаследованный дрейф — Статьи 1 и 0′
Why Agents Compromise Safety Under Pressure	Ручка «давление»; нормативный дрейф; рационализация как признак для роутера (Статья 3)
ReflAct	Каркас агента (Статья 1); рефлексия о цели как вмешательство (Статья 3)
ReAct, Reflexion	Каркасы агента; Статьи 1 и 2
AgentBench	Источник нового домена для `envgen`

7 Риски и оговорки

- **Объём.** Серия — семь статей. Для Т3 достаточно Статей 1–2 и MVP. Статьи 0, 0′, 3, 4, 5 не должны задерживать сдачу Т3.
- **Новизна проверена поверхностно.** Ссылки с пометкой [по аннотации] найдены веб-поиском и не прочитаны. Самые конкурентные места:
  - Статья 0 — Goal-Drift Probes и When Attention Closes;
  - Статья 1 — ODCV-Bench;
  - Статья 3 — Learning to Steer и Intervention Paradox;
  - Статья 5 — Learning to Steer.

  Перед каждой статьёй нужен полный поиск и чтение ближайших работ.
- **Подозрительные идентификаторы.** Часть найденных arXiv-идентификаторов не проверена по первоисточнику; до цитирования сверить.
- **Вычисления.** Статьи 0, 0′ и 5 требуют GPU: пробы на моделях до 30B, латентный стенд, RL на моделях до 9B. Статья 1 требует бюджета на API фронтиров.
- **Планка ниже 70%.** Если фронтиры не ломаются даже на максимальной дозе, это тоже результат (DD50 больше максимальной дозы), но для бенчмарка нужны дополнительные ручки или более длинные горизонты.
- **Интеграции** (Claude Code, OpenAI Agents SDK, LangGraph, vLLM, SGLang, Harbor, Inspect AI, TRL, verl, OpenTelemetry) — цели, совместимость не проверена.
- **Не учтено:** гугл-док Славы (трек 0) — добавить, когда будет доступен.

Новые ссылки этой версии (по аннотациям)

- Goal-Drift Probes (ICML 2026): https://icml.cc/virtual/2026/67962
- When Attention Closes: https://arxiv.org/abs/2605.12922
- Trait-space Monitoring for Emergent Misalignment: https://arxiv.org/abs/2606.07631
- Alignment via Training Against Probes: https://arxiv.org/abs/2609.38645
- RL-Obfuscation: https://arxiv.org/abs/2506.14261; Neural Chameleons: https://arxiv.org/abs/2512.11949
- Out of Sight, Not Out of Mind: https://arxiv.org/abs/2605.28214
- When Latent Agents Lie: https://arxiv.org/abs/2606.28958; LCGuard: https://arxiv.org/abs/2605.22786
- ODCV-Bench: https://arxiv.org/abs/2512.20798
- Evaluation Awareness and Multi-Agent Monitoring: https://arxiv.org/abs/2609.03035
- The Intervention Paradox: https://arxiv.org/abs/2602.03338
- Ctrl-Z: https://arxiv.org/abs/2504.10374; Retrying vs Resampling: https://blog.redwoodresearch.org/p/retrying-vs-resampling-in-ai-control
- AgentTether: https://arxiv.org/abs/2607.06273; LivePlan: https://arxiv.org/abs/2608.06701
- Learning to Steer LLM Agents: https://arxiv.org/abs/2610.22316; TACT: https://arxiv.org/abs/2605.05980
- Training Deliberative Monitors for Black-Box Scheming Detection: https://arxiv.org/abs/2605.29601
- AgentDoG 1.5: https://arxiv.org/abs/2605.29801
- Safety Testing LLM Agents at Scale (VERA, дообученный Qwen3Guard): https://arxiv.org/abs/2607.01793
- Evaluating LLM Calibration for Coding Agent Monitoring (Apollo Research): https://www.apolloresearch.ai/products/evaluating-llm-calibration-for-coding-agent-monitoring/
- InterventionBench: https://aclanthology.org/2026.findings-emnlp.698/; BLINDSPOT: https://arxiv.org/abs/2609.16305

- CUSUM-Shaped Monitoring: https://arxiv.org/abs/2607.20129
- Alignment Collapse Under KV Cache Quantization: https://arxiv.org/abs/2606.09864
- The Pitfalls of KV Cache Compression: https://arxiv.org/abs/2510.00231
- Governance Decay: https://arxiv.org/abs/2606.22528
- Speculative Decoding at Temperature Zero: https://arxiv.org/abs/2606.25097
- Graph-Based Drift Recovery: https://arxiv.org/abs/2608.14109
- A Closer Look at System Prompt Robustness: https://arxiv.org/abs/2502.12197
- Demystifying RL for Long-Horizon Tool-Using Agents: https://arxiv.org/abs/2603.21972
- Policy-Conditioned Counterfactual Credit: https://arxiv.org/abs/2606.05263; CRAFT: https://arxiv.org/abs/2606.29476; G2PO: https://arxiv.org/abs/2606.22995